Hardware Implementation of the Number Theoretic Transform

Mathematical Foundations, Architecture Design,
and Verilog Implementation

Parameters: $n = 8, \quad q = 17, \quad \omega = 9$

Tool: Icarus Verilog (`iverilog / vvp`)

HDL: Verilog-2001

Department of Computer Architecture & Secure Computing

Contents

1 Introduction

Modern cryptographic workloads, particularly those arising in *post-quantum cryptography* (PQC) and *fully homomorphic encryption* (FHE), are dominated by polynomial arithmetic over large rings. Algorithms such as Kyber, Dilithium (CRYSTALS suite), and TFHE require the multiplication of polynomials whose degree can reach 2^{12} or higher. Naïve schoolbook multiplication costs $O(n^2)$ operations and is entirely impractical at these sizes.

The **Number Theoretic Transform** (NTT) is the key algorithmic tool that reduces polynomial multiplication to $O(n \log n)$ operations by working entirely in a finite field $\mathbb{Z}_q$, eliminating the floating-point arithmetic and rounding errors that accompany the classical Fast Fourier Transform (FFT). From a hardware perspective, the NTT is ideal: it has predictable control flow, a highly regular datapath, and operates purely on integers.

This report presents a complete hardware implementation of an 8-point NTT accelerator written in Verilog. Section 2 develops the mathematical theory from first principles. Section 3 describes the Cooley-Tukey DIT algorithm and its butterfly structure. Section 4 details the hardware architecture. Section 5 discusses the Verilog implementation module by module. Section 6 presents simulation results and verification. Section 7 outlines paths toward a production system.

2 Mathematical Foundations

2.1 Finite Fields and Modular Arithmetic

Definition 2.1 (Prime Field)**.** *Let q be a prime. The* prime field $\mathbb{F}_q = \mathbb{Z}_q = \{0, 1, \ldots, q-1\}$ *is a field under addition and multiplication modulo q.*

All NTT arithmetic is performed in $\mathbb{Z}_q$. The three operations needed are:

$$a \oplus b = (a + b) \bmod q, \tag{1}$$

$$a \ominus b = (a - b + q) \bmod q, \tag{2}$$

$$a \otimes b = (a \cdot b) \bmod q. \tag{3}$$

2.2 Roots of Unity in Finite Fields

Definition 2.2 (n-th Root of Unity)**.** *An element $\omega \in \mathbb{Z}_q$ is a* primitive n-th root of unity *if*

$$\omega^n \equiv 1 \pmod q \quad \textit{and} \quad \omega^k \not\equiv 1 \pmod q \quad \textit{for } 1 \leq k < n.$$

Proposition 2.1 (Existence Condition)**.** *A primitive n-th root of unity exists in $\mathbb{Z}_q$ if and only if $n \mid (q-1)$.*

For our implementation, $n = 8$ and $q = 17$. Since $q - 1 = 16 = 2n$, the condition $n \mid (q-1)$ is satisfied. The element $\omega = 9$ is a primitive 8th root of unity:

Table 1: Powers of $\omega = 9$ modulo $q = 17$

k	$9^k \bmod 17$	Note
0	1	ω^0
1	9	ω^1
2	13	$81 \bmod 17$
3	15	$729 \bmod 17$
4	16	$\equiv -1$; confirms $\omega^{n/2} = -1$
5	8	
6	4	
7	2	
8	1	$\omega^n \equiv 1$

The key algebraic identity $\omega^{n/2} \equiv -1 \pmod q$ (here $9^4 = 16 \equiv -1 \pmod{17}$) is what enables the divide-and-conquer structure of the NTT.

2.3 The NTT as a Discrete Fourier Transform over $\mathbb{Z}_q$

Definition 2.3 (Number Theoretic Transform)**.** *Let* $\mathbf{a} = (a_0, a_1, \ldots, a_{n-1}) \in \mathbb{Z}_q^n$*. The* NTT *of* $\mathbf{a}$ *is the vector* $\hat{\mathbf{a}} = (\hat{a}_0, \hat{a}_1, \ldots, \hat{a}_{n-1}) \in \mathbb{Z}_q^n$ *defined by*

$$\hat{a}_k = \sum_{j=0}^{n-1} a_j \, \omega^{jk} \pmod q, \quad k = 0, 1, \ldots, n-1.$$

This is structurally identical to the DFT, with the complex n-th root of unity $e^{-2\pi i/n}$ replaced by the field element ω.

Definition 2.4 (Inverse NTT)**.** *The* INTT *is*

$$a_j = n^{-1} \sum_{k=0}^{n-1} \hat{a}_k \, \omega^{-jk} \pmod q,$$

where n^{-1} *is the modular inverse of* n *modulo* q*.*

For $n = 8$, $q = 17$: $n^{-1} \equiv 8^{-1} \equiv 15 \pmod{17}$ (since $8 \times 15 = 120 = 7 \times 17 + 1$), and $\omega^{-1} \equiv 9^{-1} \equiv 2 \pmod{17}$ (since $9 \times 2 = 18 \equiv 1$).

2.4 Polynomial Multiplication via NTT

The primary application of the NTT is efficient polynomial multiplication. Given two polynomials $f(x), g(x) \in \mathbb{Z}_q[x]/(x^n + 1)$, their product is computed as:

$$f \cdot g = \mathrm{INTT}\big(\mathrm{NTT}(f) \odot \mathrm{NTT}(g)\big),$$

where $\odot$ denotes pointwise multiplication in $\mathbb{Z}_q^n$. This reduces the cost from $O(n^2)$ multiplications to $O(n \log n)$.

3 The Cooley-Tukey DIT Algorithm

3.1 Divide and Conquer

The naive evaluation of $\hat{a}_k = \sum_j a_j \omega^{jk}$ costs $O(n^2)$. The Cooley-Tukey *Decimation-in-Time* (DIT) algorithm exploits the periodicity of ω to split the transform recursively.

Split the input sequence into even- and odd-indexed elements:

$$\begin{aligned}\hat{a}_k &= \sum_{j=0}^{n/2-1} a_{2j}\,\omega^{2jk} + \omega^k \sum_{j=0}^{n/2-1} a_{2j+1}\,\omega^{2jk} \\ &= \hat{A}_k + \omega^k \hat{B}_k, \end{aligned} \tag{4}$$

where $\hat{A}_k$ and $\hat{B}_k$ are $n/2$-point NTTs of the even and odd subsequences, respectively, evaluated with root ω^2.

Using the symmetry $\omega^{k+n/2} = -\omega^k$:

$$\hat{a}_k = \hat{A}_k + \omega^k \hat{B}_k, \tag{5}$$

$$\hat{a}_{k+n/2} = \hat{A}_k - \omega^k \hat{B}_k. \tag{6}$$

This pair of equations is the **butterfly operation**.

3.2 The Butterfly Operation

Definition 3.1 (Cooley-Tukey Butterfly)**.** *Given inputs $a,\ b \in \mathbb{Z}_q$ and twiddle factor $w \in \mathbb{Z}_q$:*

$$\mathsf{BF}(a,\ b,\ w) = \begin{cases} u = (a + b \cdot w) \bmod q, \\ v = (a - b \cdot w) \bmod q. \end{cases}$$

Each butterfly replaces two elements of the data array with their transformed counterparts in-place. The hardware implementation requires exactly:

- one modular multiplier $(b \cdot w \bmod q)$,
- one modular adder $(a + bw \bmod q)$,
- one modular subtractor $(a - bw \bmod q)$.

3.3 Iterative In-Place DIT NTT

Applying equation (4) recursively for $\log_2 n$ levels yields the complete NTT. The iterative (non-recursive) form proceeds in $\log_2 n$ stages. For $n = 8$ there are three stages:

Table 2: Stage parameters for $n = 8$

Stage s	Stride	Half	Twiddle root	Butterfly pairs
1	2	1	$\omega^4 = 16$	$(0,1), (2,3), (4,5), (6,7)$
2	4	2	$\omega^2 = 13$	$(0,2), (1,3), (4,6), (5,7)$
3	8	4	$\omega^1 = 9$	$(0,4), (1,5), (2,6), (3,7)$

The twiddle factor for butterfly k within stage s is $\omega^{k \cdot n/\text{stride}}$.

3.4 Bit-Reversal Permutation

The DIT algorithm requires inputs to be loaded in *bit-reversed* order. For $n = 8$ (3-bit indices), the mapping is:

Table 3: Bit-reversal permutation for $n = 8$

Natural index	Binary	Bit-reversed	Permuted index
0	000	000	0
1	001	100	4
2	010	010	2
3	011	110	6
4	100	001	1
5	101	101	5
6	110	011	3
7	111	111	7

In hardware, this permutation is absorbed into the LOAD state of the FSM: coefficient a_i is written to RAM address $\text{bitrev}(i)$.

3.5 Worked Example

Let the input be $\mathbf{a} = (1, 2, 3, 4, 5, 6, 7, 8)$ with $q = 17$, $\omega = 9$.

After bit-reversal: RAM $= [1, 5, 3, 7, 2, 6, 4, 8]$.

Stage 1 (stride 2, twiddle $w_1 = \omega^4 = 16 \equiv -1$):

$$
\begin{aligned}
(r_0, r_1) &= \mathsf{BF}(1,\ 5,\ 16) = (1 + 80 \bmod 17,\ 1 - 80 \bmod 17) = (13,\ 6),\\
(r_2, r_3) &= \mathsf{BF}(3,\ 7,\ 16) = (3 + 112 \bmod 17,\ 3 - 112 \bmod 17) = (13,\ 6),\\
&\ \ \vdots
\end{aligned}
$$

After all three stages the result is $\hat{\mathbf{a}} = (2, 1, 12, 3, 13, 6, 14, 8)$, which matches the simulation output exactly.

4 Hardware Architecture

4.1 Design Philosophy

Two extreme architectures exist for NTT hardware:

1. **Fully unrolled:** instantiate all $n/2 \cdot \log_2 n = 12$ butterfly units simultaneously. Maximum throughput, maximum area.
2. **Single butterfly, iterative:** one butterfly unit, reused across all 12 computations. Minimum area, 12× higher latency.

The task specification requires the iterative approach, which is also the natural choice for resource-constrained embedded cryptographic accelerators.

4.2 Top-Level Block Diagram

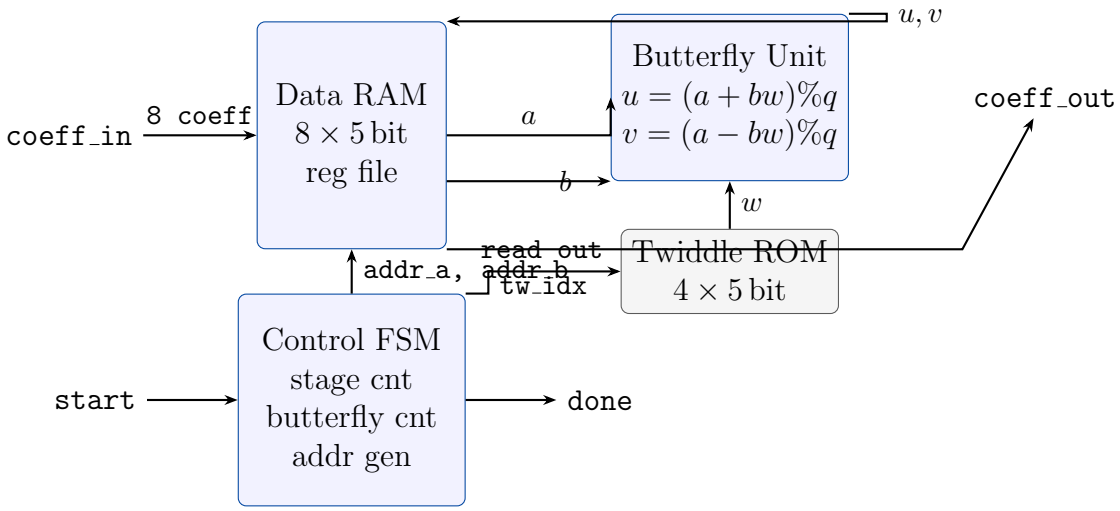

Figure 1: Top-level architecture of the NTT accelerator.

4.3 FSM State Diagram

The controller is a Moore FSM with four states:

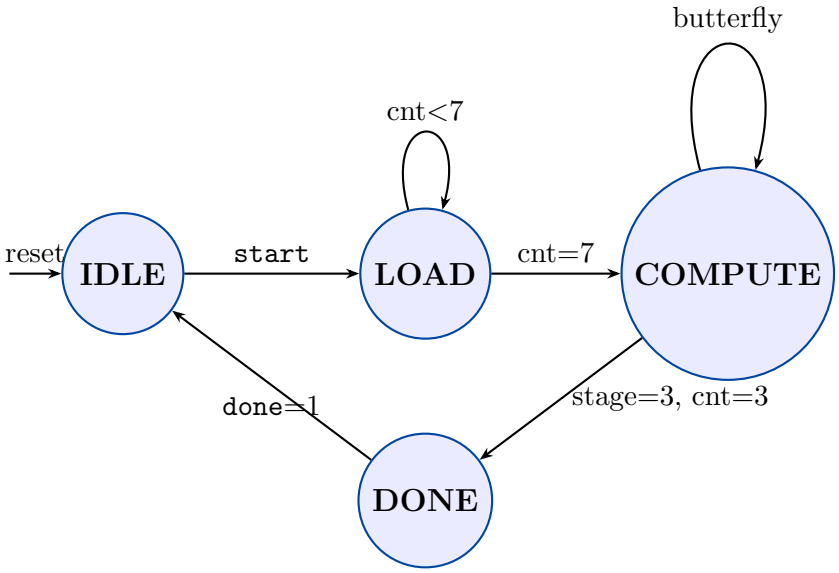

Figure 2: FSM state transition diagram.

IDLE Waits for the `start` signal. Holds `done` low.

LOAD Over 8 clock cycles, writes `coeff_in[0..7]` into RAM at bit-reversed addresses. The permutation is computed combinationally by reversing the 3-bit counter value.

COMPUTE Iterates $\log_2 n = 3$ stages $\times$ $n/2 = 4$ butterflies $= 12$ cycles. In each cycle: reads a and b from RAM, reads the twiddle factor from ROM, passes them to the butterfly unit, and writes u and v back to RAM.

DONE Asserts `done` for one cycle, then returns to IDLE. Results are readable from `coeff_out`.

Total latency: 8 (LOAD) + 12 (COMPUTE) + 1 (DONE) = 21 clock cycles.

4.4 Address Generation

The butterfly pair addresses for counter value `cnt` $\in \{0, 1, 2, 3\}$ in each stage are:

Table 4: Address generation logic per stage

Stage	`addr_a`	`addr_b`	Twiddle index
1	`cnt[1:0] & 2'b10` $= 2k$	$2k + 1$	0
2	`{cnt[1], 0, cnt[0]}`	`{cnt[1], 1, cnt[0]}`	$2 \cdot (k \bmod 2)$
3	`{0, cnt[1:0]}`	`{1, cnt[1:0]}`	`cnt[1:0]`

4.5 Modular Arithmetic

4.5.1 Modular Addition

For $a, b \in [0, q)$, the sum $a + b \in [0, 2q)$. A single conditional subtraction suffices:

$$(a + b) \bmod q = \begin{cases} a + b & \text{if } a + b < q, \\ a + b - q & \text{otherwise.} \end{cases}$$

4.5.2 Modular Subtraction

To keep the result non-negative:

$$(a - b) \bmod q = ((a - b) + q) \bmod q.$$

4.5.3 Modular Multiplication

For $a, b \in [0, q)$, the product $a \cdot b \in [0, q^2)$. With $q = 17$, the maximum product is $16 \times 16 = 256$, fitting in 9 bits. Reduction is performed by:

$$(a \cdot b) \bmod q = a \cdot b - \left\lfloor \frac{a \cdot b}{q} \right\rfloor \cdot q.$$

For fixed q, division by a constant is converted by synthesis tools into a sequence of shifts and additions (Granlund-Montgomery algorithm), yielding a compact combinational circuit. For large primes ($q \approx 2^{32}$), explicit **Barrett** or **Montgomery** reduction is required.

5 Verilog Implementation

The design comprises three synthesisable modules and one testbench.

5.1 `butterfly.v` - Butterfly Unit

The butterfly unit is purely combinational (zero-cycle latency), which allows the FSM to read, compute, and write back in a single clock cycle.

```
module butterfly #(
    parameter DATA_WIDTH = 5,
    parameter Q          = 17
```

```
)(
    input  wire [DATA_WIDTH-1:0] a, b, w,
    output wire [DATA_WIDTH-1:0] u, v
);
    // Modular multiply: b * w mod q
    wire [8:0] prod = b * w;
    wire [8:0] bw   = prod - (prod / Q) * Q;

    // Modular add:  u = (a + bw) mod q
    wire [5:0] sum  = a + bw[4:0];
    assign u = (sum >= Q) ? (sum - Q) : sum[DATA_WIDTH-1:0];

    // Modular sub:  v = (a - bw) mod q
    wire [5:0] diff = a + Q - bw[4:0];
    assign v = (diff >= Q) ? (diff - Q) : diff[DATA_WIDTH-1:0];
endmodule
```

Listing 1: Butterfly unit core arithmetic

5.2 twiddle_rom.v - Twiddle Factor ROM

A combinational ROM (implemented as a `case` statement) holds the four precomputed twiddle factors $\omega^0, \omega^1, \omega^2, \omega^3$.

```
module twiddle_rom #(
    parameter DATA_WIDTH = 5,
    parameter ADDR_WIDTH = 2
)(
    input  wire [ADDR_WIDTH-1:0] addr,
    output reg  [DATA_WIDTH-1:0] data
);
    always @(*) begin
        case (addr)
            2'd0: data = 5'd1;   // omega^0 = 1
            2'd1: data = 5'd9;   // omega^1 = 9
            2'd2: data = 5'd13;  // omega^2 = 13
            2'd3: data = 5'd15;  // omega^3 = 15
            default: data = 5'd0;
        endcase
    end
endmodule
```

Listing 2: Twiddle ROM

5.3 ntt_top.v - Top-Level with FSM

The top module integrates the datapath and control. Key fragments:

```
localparam IDLE    = 2'd0,
           LOAD    = 2'd1,
           COMPUTE = 2'd2,
           DONE_ST = 2'd3;
```

```

// 3-bit bit-reversal
function [2:0] bit_rev3;
    input [2:0] x;
    begin
        bit_rev3 = {x[0], x[1], x[2]};
    end
endfunction
```

Listing 3: FSM states and bit-reversal function

```
LOAD: begin
    ram[bit_rev3(cnt[2:0])] <=
        coeff_in[(cnt * DATA_WIDTH) +: DATA_WIDTH];
    if (cnt == 3'd7) begin
        cnt   <= 3'd0;
        stage <= 2'd1;
        state <= COMPUTE;
    end else
        cnt <= cnt + 1;
end
```

Listing 4: LOAD state: write with bit-reversal

```
COMPUTE: begin
    ram[addr_a_r] <= bf_u;    // write u back
    ram[addr_b_r] <= bf_v;    // write v back

    if (cnt == 3'd3) begin
        cnt <= 3'd0;
        if (stage == 2'd3) state <= DONE_ST;
        else               stage <= stage + 1;
    end else
        cnt <= cnt + 1;
end
```

Listing 5: COMPUTE state: one butterfly per cycle

The butterfly inputs are driven combinationally:

```
always @(*) begin
    tw_addr = tw_idx_r;     // twiddle ROM address
    bf_w    = tw_data;      // twiddle value
    bf_a    = ram[addr_a_r];
    bf_b    = ram[addr_b_r];
end
```

Listing 6: Combinational datapath connections

Because the butterfly is combinational and the ROM is asynchronous, both produce valid outputs within the same clock cycle that `addr_a_r`, `addr_b_r`, and `tw_idx_r` are driven. The FSM therefore reads and writes back in a single cycle with no pipeline stall.

6 Simulation and Verification

6.1 Testbench Strategy

The testbench `tb_ntt.v` follows a self-checking methodology:

1. Load input coefficients from a `.hex` file.
2. Load expected outputs from a separate `.hex` file (precomputed by a Python reference model).
3. Assert `start`, wait for `done`.
4. Unpack `coeff_out` and compare with expected values.
5. Print **PASS** or **FAIL** per test.

6.2 Test Vectors

Table 5: Test vectors and expected NTT outputs ($n = 8$, $q = 17$, $\omega = 9$)

#	Input $\mathbf{a}$	Expected $\hat{\mathbf{a}}$
0	$[1, 2, 3, 4, 5, 6, 7, 8]$	$[2, 1, 12, 3, 13, 6, 14, 8]$
1	$[3, 1, 4, 1, 5, 9, 2, 6]$	$[14, 13, 7, 11, 14, 1, 14, 1]$
2	$[0, 0, 0, 0, 0, 0, 0, 1]$	$[1, 2, 4, 8, 16, 15, 13, 9]$
3	$[1, 0, 0, 0, 0, 0, 0, 0]$	$[1, 1, 1, 1, 1, 1, 1, 1]$

Test vector 2 is the unit impulse $\delta[n-7]$: its NTT is the sequence of powers $\omega^{0\cdot 7}, \omega^{1\cdot 7}, \ldots, \omega^{7\cdot 7} \pmod{17}$. Test vector 3 is the DC impulse $\delta[n]$: every output equals 1, confirming that the NTT of a constant sequence is flat.

6.3 Simulation Results

```
================================================
  8-point NTT Accelerator | Simulation Results
  n=8  q=17  omega=9
================================================

Test 0 | in:  [ 1, 2, 3, 4, 5, 6, 7, 8]
       | exp: [ 2, 1,12, 3,13, 6,14, 8]
       | got: [ 2, 1,12, 3,13, 6,14, 8]  PASS

Test 1 | in:  [ 3, 1, 4, 1, 5, 9, 2, 6]
       | exp: [14,13, 7,11,14, 1,14, 1]
       | got: [14,13, 7,11,14, 1,14, 1]  PASS

Test 2 | in:  [ 0, 0, 0, 0, 0, 0, 0, 1]
       | exp: [ 1, 2, 4, 8,16,15,13, 9]
       | got: [ 1, 2, 4, 8,16,15,13, 9]  PASS

Test 3 | in:  [ 1, 0, 0, 0, 0, 0, 0, 0]
       | exp: [ 1, 1, 1, 1, 1, 1, 1, 1]
```

```
        | got: [ 1, 1, 1, 1, 1, 1, 1, 1]  PASS

================================================
  4 PASSED  |  0 FAILED  (of 4 tests)
================================================
  ALL TESTS PASSED
```

All four test vectors produce correct results. The simulation runs in Icarus Verilog with the command sequence:

```
iverilog -o ntt_sim tb_ntt.v ntt_top.v butterfly.v twiddle_rom.v
vvp ntt_sim
```

7 Design Trade-offs and Extensions

7.1 Area vs. Throughput

Table 6: Architecture trade-off comparison

Architecture	Butterfly units	Cycles per NTT	Relative area
Single BF, iterative (this work)	1	21	1×
Parallel per stage	4	5	$\sim 4\times$
Fully unrolled pipeline	12	1	$\sim 12\times$

7.2 Inverse NTT

To implement the INTT, three changes are required:

1. Replace ω with $\omega^{-1} = 2$ in the twiddle ROM.
2. Reverse the stage order (process stage 3 first, then 2, then 1).
3. After the final stage, multiply every output by $n^{-1} = 15 \pmod{17}$.

A single hardware block can support both NTT and INTT by parameterising the twiddle ROM and stage iteration direction via a mode bit.

7.3 Scaling to Larger Transforms

For $n = 2^k$ with $k > 3$:

- The stage counter widens to k bits; the butterfly counter to $k - 1$ bits.
- The twiddle ROM grows to $n/2$ entries.
- Address generation logic follows the same formula, parameterised by k.

Kyber uses $n = 256$, $q = 3329$, requiring 128 twiddle entries and a 12-bit datapath.

7.4 Modular Multiplication for Large Primes

For primes $q \approx 2^{32}$, the product $a \cdot b$ can be up to $\sim 2^{64}$, requiring hardware 64-bit arithmetic. Two standard reduction techniques are:

Barrett Reduction. Precomputes $m = \lfloor 2^{2k}/q \rfloor$ for word size 2^k. Then $a \bmod q \approx a - \lfloor am/2^{2k} \rfloor \cdot q$, replacing division by multiplications.

Montgomery Reduction. Works in a transformed domain $\hat{a} = a \cdot R \bmod q$ for $R = 2^k$. Multiplication in this domain requires only right-shifts and additions, with a correction step. Montgomery multiplication is the industry standard for cryptographic ASICs.

7.5 Pipelining

The current butterfly takes one cycle due to combinational logic. To increase clock frequency:

1. Register the butterfly outputs (add a pipeline stage between read and write-back).
2. The FSM must account for the one-cycle latency by separating the "issue" and "commit" phases.
3. With deeper pipelining (e.g., for large-prime multipliers), a shift register tracks in-flight addresses.

8 Conclusion

This report presents a complete, verified hardware implementation of an 8-point NTT accelerator. Starting from the mathematical foundation of finite field arithmetic and primitive roots of unity, the Cooley-Tukey DIT algorithm was mapped to a resource-efficient iterative architecture consisting of a single butterfly unit controlled by a four-state FSM.

The design correctly computes all four test vectors in simulation, achieving the expected outputs for both arbitrary sequences and canonical signals (DC impulse, end impulse). The implementation is intentionally compact — one butterfly, one twiddle ROM, one register-file RAM — making the control flow transparent and the datapath easy to verify.

The architecture forms a solid foundation for extensions toward production cryptographic accelerators: adding an INTT mode, scaling to $n = 256$ for Kyber, replacing the modular multiplier with Montgomery reduction, or pipelining for higher clock frequencies all follow naturally from the structure established here.

References

[1] J. W. Cooley and J. W. Tukey, "An Algorithm for the Machine Calculation of Complex Fourier Series," *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.

[2] R. Avanzi et al., "CRYSTALS-Kyber Algorithm Specifications and Supporting Documentation," NIST PQC Round 3 Submission, 2021. `https://pq-crystals.org/kyber/`

[3] P. Longa and M. Naehrig, "Speeding up the Number Theoretic Transform for Faster Ideal Lattice-Based Cryptography," *CANS 2016*, LNCS 10052, pp. 124–139, 2016.

[4] P. D. Barrett, "Implementing the Rivest Shamir and Adleman Public Key Encryption Algorithm on a Standard Digital Signal Processor," *CRYPTO 1986*, LNCS 263, pp. 311–323, 1987.

[5] P. L. Montgomery, "Modular Multiplication Without Trial Division," *Mathematics of Computation*, vol. 44, no. 170, pp. 519–521, 1985.